Dark Pattern Auditor — TCS GPT-4o Client/Server Architecture

How one Stage-2 LLM call travels from auto_audit.py, through the TCS code-server proxy, to GPT-4o, and back.

CLIENT — this repo (runs wherever you execute dpa.py)

auto_audit.py / record.py --auto-audit

CLI entry point. Preflight-checks required credentials for the active backend before spending anything (`llm/backends/factory.py::preflight_message()`).

↓

llm/api_client.py

`call_for_judgement(payload_dir, model, ...)`

Enforces the per-episode LLM call budget first, *before* any network call. Then dispatches by `LLM_BACKEND` — an explicit `client=` kwarg always forces the legacy Anthropic path (test seam), otherwise reads the resolved backend.

↓

llm/backends/factory.py → TCSGPT4oBackend

`llm/backends/tcs_gpt4o_backend.py`

Reads `TCS_LLM_URL` / `TCS_SESSION_COOKIE` from `.env` . Builds the multipart request: `prompt_id, text, system, images[], response_schema, temperature, seed, max_tokens` .

↓

llm/image_tiling.py + request handling

Splits tall full-page screenshots into ~1280×2000px overlapping tiles, each labeled by page + position in the request text (GPT-4o downscales large images until text is unreadable — Claude's 8000px budget-based path doesn't need this).

- Retries on **5xx / timeout only**, exponential backoff (never on 4xx)
- Non-JSON response body → `TcsSessionExpiredError` (expired cookie)
- Malformed JSON output → one repair retry, then fails loudly
- Cookie sent via `cookies=` , never logged

HTTPS via the code-server proxy
`POST {TCS_LLM_URL}/execute`
`Cookie: code-server-session=…`

↔

SERVER — server/ (runs inside the TCS dev workspace, port 8000)

https://rniazure.tcsapps.com/dev-workspaces/<id>/proxy/8000

TCS's code-server reverse proxy — authenticates the inbound cookie, forwards to `localhost:8000` inside the workspace.

↓

server/app.py (FastAPI)

`POST /execute · GET /health`

Thin relay — validates image count/size, parses `response_schema` , and relays to GPT-4o. Does **not** store prompts/text/images; only `prompt_id` is logged.

↓

server/openai_client.py

Picks the transport by which env vars are set:

- `AZURE_OPENAI_ENDPOINT` present → Azure OpenAI
- else `OPENAI_API_KEY` present → OpenAI-compatible endpoint

Requests structured output via `response_format: json_schema (strict)` ; falls back to `json_object` if the deployment rejects it.

↓

GPT-4o

Returns the completion. Response relayed back as: `ok, output_text, output_json, model, system_fingerprint, usage{prompt_tokens, completion_tokens}, latency_ms, request_id, error` .

Alternate path — LLM_BACKEND=anthropic

Bypasses server/ and the TCS proxy entirely: `llm/backends/anthropic_backend.py` calls `anthropic.Anthropic().messages.create(...)` directly with `ANTHROPIC_API_KEY`. Used for comparison runs against the original Claude results, or as a fallback if the TCS workspace is unavailable.

Every call is logged to `payload/<episode_id>/…/usage.json` (tokens, latency, estimated cost) and aggregated per-episode via `dpa.py cost-report` (`llm/cost_aggregate.py`), for comparison against Sun et al. 2026's reported ~287k tokens / $0.52 per site.